

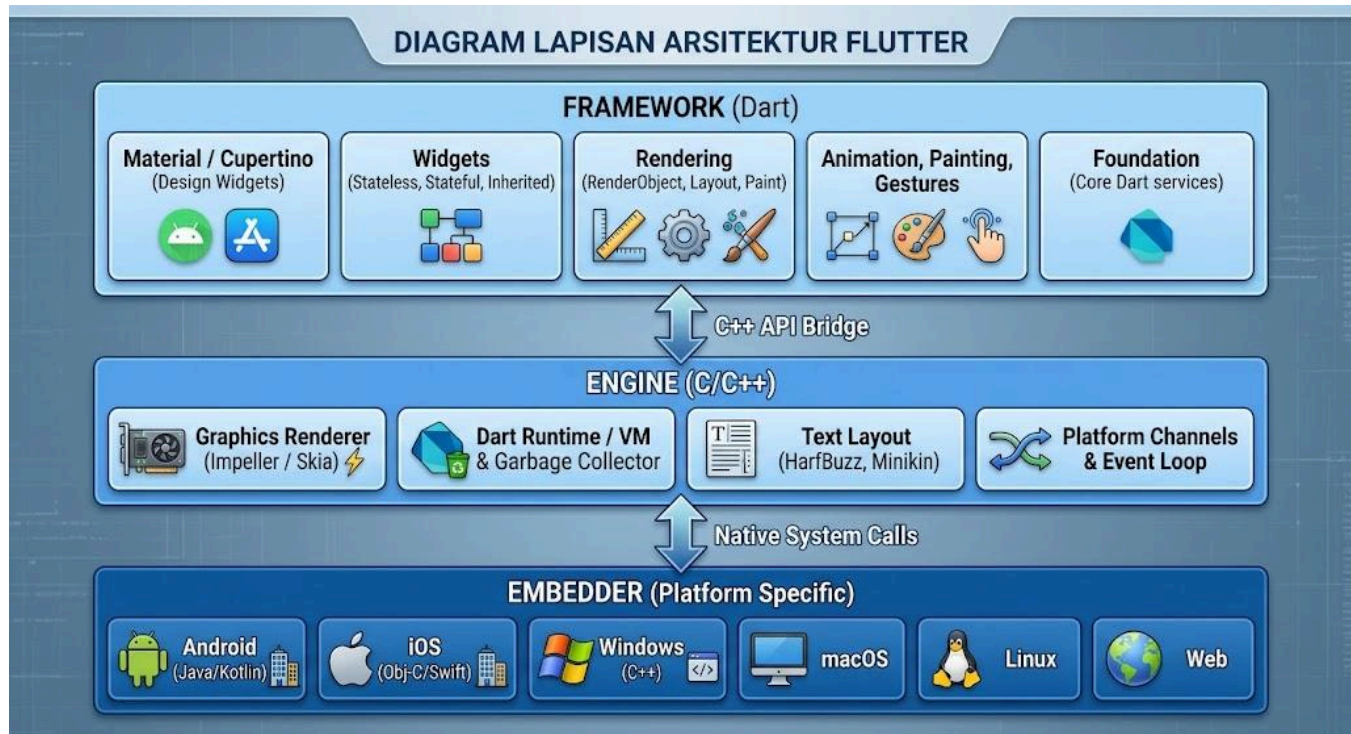
Dasar Dart

Dosen	: IKetut Gunawan, S.KOM, M.T.I
Kode/Mata Kuliah	: KB1185/ Aplikasi Mobile
Bobot Mata Kuliah	: 3 Sks
Prodi	: Teknik Informatika
Pertemuan	: 2

Cakupan Materi

- Pengenalan Lingkungan Kerja Cloud: DartPad
 - Sistem Tipe Data Eksplisit & Sound Null Safety
 - Immutability (**final** vs **const**) & Lifecycle Variabel (**late**)
 - Type Data Sederhana (**List**), String Interpolation
- Assignment: Setup Environment Flutter Mandiri (Homework) + Commit Code Latihan

Arsitektur Flutter



1

Pengenalan Lingkungan Kerja: DartPad

Mengapa DartPad dengan DartPad ?

difokuskan pada penguasaan sintaks, alur data, dan paradigma bahasa Dart secara instan

Akses Resmi: Buka browser dan jalankan <https://dartpad.dev/>

Fitur Utama: Eksekusi kode instan berbasis cloud, formatting otomatis (*Auto-Format*), serta pembuatan tautan proyek (*Share*).

Batasan DartPad:

- **Single File Only:** Seluruh deklarasi class, fungsi pembantu, dan fungsi `main()` harus berada dalam satu editor layar yang sama.
- **Tanpa Akses Hardware:** Tidak dapat mengakses media penyimpanan file lokal, kamera, atau sensor fisik komputer.
- **Ketergantungan Jaringan:** Membutuhkan koneksi internet untuk mengompilasi kode pada server Google

2

Sistem Tipe Data & Sound Null Safety

2.1. Explicit Typing (Tanpa var):

Deklarasi tipe data dasar (`int`, `double`, `String`, `bool`, `DateTime`) secara eksplisit.

Ganti `var` langsung dengan nama tipe datanya (`int`, `double`, `String`, `bool`, `List`, dll). Variabel ini tetap dapat diubah nilainya (*mutable*), asalkan tipe datanya konsisten.

```
String productName = 'Kopi Susu';
int stock = 15;
double price = 18000.0;
bool isAvailable = true;

// Nilai dapat diubah
stock = 20;
// stock = 'dua puluh'; // Error! Tipe data tidak boleh berubah.
```

Lakukan praktik code ini pada dartPad, nama variable tidak boleh sama dan juga valuenya dan lihat hasilnya

2.2. Sound Null Safety

Dart menjamin variabel non-nullable tidak akan pernah bernilai null pada saat runtime, mencegah galat fatal (`NullPointerException` / crash aplikasi).

- **Non-Nullable (Bawaan Default):**

```
String itemName = 'Kopi Hitam'; //
// itemName = null; // Ditolak oleh kompiler!
```

- **Nullable (?):** Berikan tanda tanya jika suatu data memang diizinkan kosong/belum tersedia:

```
String? customerNote; // Boleh berisi String atau null
customerNote = 'Kurang manis'; //
customerNote = null; // Sah
```

- **Null-Aware Operator (??):** Menyediakan nilai alternatif (*fallback*) jika data bernilai `null`:

```
String noteToPrint = customerNote ?? 'Tidak ada catatan khusus'; //
```

- **Null Assertion Operator (!):** Memaksa kompiler membaca variabel nullable sebagai non-nullable. Hindari pemakaian ini jika tidak ada jaminan 100% bahwa nilainya tidak kosong:

```
// Berisiko crash jika customerNote ternyata null: //
print(customerNote!.toUpperCase()); //
```

Summary :

Kode	Arti
<code>customerNote!</code>	Saya yakin nilainya tidak null; crash jika ternyata null
<code>customerNote?.toUpperCase()</code>	Ubah menjadi huruf kapital hanya jika tidak null
<code>customerNote ?? "Tidak ada catatan"</code>	Gunakan nilai pengganti jika null

2.2. final dengan Tipe Eksplisit (Single Assignment / Runtime Constant)

Nilai hanya dapat diisi satu kali saat aplikasi berjalan (*runtime*), setelah itu terkunci dan tidak dapat diubah lagi.

```
final String transactionId = 'TRX-9901';
final DateTime transactionTime = DateTime.now();

// Menuliskan tipe data setelah 'final' bersifat opsional tapi sangat
disarankan:
// final transactionId = 'TRX-9901'; // Valid, tapi implisit
```

2.3. const dengan Tipe Eksplisit (Compile-Time Constant)

Nilai harus sudah diketahui sebelum program dijalankan (*compile-time*). Nilai ini bersifat absolut dan tidak bergantung pada input runtime.

```
const double taxRate = 0.11;
const String appCurrency = 'IDR';
```

2.5. late Modifier (Inisialisasi Tertunda)

Gunakan jika tipe data non-nullable, tetapi nilainya baru bisa ditentukan setelah deklarasi (sebelum pertama kali dibaca).

```
late String receiptNumber;

void generateReceipt() {
    receiptNumber = 'REC-${DateTime.now().millisecondsSinceEpoch}';
    print(receiptNumber); // Aman dibaca setelah diinisialisasi
}
```

Ringkasan Sintaks

<i>Pola Deklarasi</i>	<i>Karakteristik</i>	<i>Contoh</i>
Type variableName = value;	Nilai bisa diubah, tipe data terkunci mutlak	int quantity = 2;
final Type variableName = value;	Nilai dikunci saat runtime	final double total = price * quantity;
const Type variableName = value;	Nilai dikunci saat compile-time	const int maxItemLimit = 50;
Type? variableName;	Boleh berisi nilai atau null	String? promoCode;

2.6. Daftar Type Data

2.6.1. String — data teks

Digunakan untuk menyimpan nama, alamat, pesan, email, dan data berbentuk teks.

```
String nama = 'Budi';  
String alamat = 'Tangerang';  
  
print(nama.toUpperCase());  
  
//Result ?
```

Dalam Flutter, **String** sering digunakan pada widget **Text**:

```
String namaProduk = 'Botol Plastik';  
Text(namaProduk);
```

String interpolation digunakan untuk menggabungkan variabel dengan teks:

```
String nama = 'Budi';  
int umur = 20;  
  
Text('Nama saya $nama, umur saya $umur tahun');  
//Result ?
```

Gunakan **String** karena widget **Text** membutuhkan data berupa teks.

2.6.2. int — bilangan bulat

Digunakan untuk bilangan tanpa angka desimal.

```
int umur = 20;  
int jumlahBarang = 5;  
int poin = 1000;
```

Contoh dalam Flutter:

```
int jumlahBarang = 5;  
  
Text('Jumlah barang: $jumlahBarang');
```


int cocok untuk ?

2.6.3. *double* — *bilangan desimal*

Digunakan untuk angka yang mempunyai bagian pecahan atau desimal.

```
double berat = 2.5;  
double harga = 15000.50;  
double rating = 4.8;
```

Contoh dalam Flutter:

```
double beratSampah = 2.5;  
Text('Berat: $beratSampah kg');
```

Gunakan **double** untuk ?

2.6.4. *num* — *bilangan umum*

num dapat menyimpan **int** maupun **double**.

```
num nilai = 10;  
nilai = 10.5;
```

Hubungan tipe bilangan:

```
num  
├── int  
└── double
```

Gunakan **num** jika sebuah nilai memang dapat berupa bilangan bulat maupun desimal. Jika bentuk datanya sudah diketahui, lebih baik gunakan **int** atau **double** agar maksud kode lebih jelas.

2.6.5. *bool* — *nilai benar atau salah*

bool hanya mempunyai dua kemungkinan:

true
false

Contoh:

```
bool sudahLogin = true;  
bool notifikasiAktif = false;
```

Contoh dalam Flutter:

```
bool isLoading = true;  
  
Widget build(BuildContext context) {  
  return isLoading  
    ? const CircularProgressIndicator()  
    : const Text('Data berhasil dimuat');  
}
```

Gunakan **bool** untuk status yang hanya memiliki dua keadaan:

- Aktif atau tidak aktif
- Berhasil atau gagal
- Sudah login atau belum
- Loading atau tidak
- Dipilih atau tidak

Berikan contoh apa yang dilakukan di sistem real world ?

2.6.6. List — kumpulan data berurutan

List digunakan untuk menyimpan banyak data dengan tipe yang sama.

```
List<String> namaMahasiswa = [  
  'Andi',  
  'Budi',  
  'Citra',  
];
```

Mengambil data berdasarkan posisi indeks:

```
print(namaMahasiswa[0]); // Andi  
print(namaMahasiswa[1]); // Budi
```

Indeks **List** dimulai dari 0.

Menambah data:

```
namaMahasiswa.add('Dewi');
```

Contoh dalam Flutter:

```
List<String> produk = [
  'Botol Plastik',
  'Kardus',
  'Minyak Jelantah',
];

ListView.builder(
  itemCount: produk.length,
  itemBuilder: (context, index) {
    return ListTile(
      title: Text(produk[index]),
    );
  },
);
```

Gunakan **List** untuk:

- Daftar produk
- ?
- ?

2.6.7. Set — kumpulan data unik

Set mirip **List**, tetapi tidak menyimpan nilai duplikat.

```
Set<String> kategori = {
  'Plastik',
  'Kertas',
  'Plastik',
};

print(kategori);
```

Hasilnya hanya:

.....

Contoh penggunaan:

```
Set<String> pilihanKategori = {};  
  
pilihanKategori.add('Plastik');  
pilihanKategori.add('Plastik');
```

Nilai **Plastik** hanya akan tersimpan sekali.

Gunakan **Set** ketika:

- Nilai tidak boleh duplikat.
- Menyimpan pilihan kategori.
- Menyimpan ID item yang sudah dipilih.
- Memeriksa keanggotaan data dengan cepat.

8. Map — data dalam bentuk key dan value

Map menyimpan data sebagai pasangan kunci dan nilai.

```
Map<String, dynamic> mahasiswa = {  
  'nama': 'Budi',  
  'umur': 20,  
  'aktif': true,  
};
```

Mengambil data:

```
print(mahasiswa['nama']); // Budi  
print(mahasiswa['umur']); // 20
```

Contoh data dari JSON atau API:

```
Map<String, dynamic> produkJson = {  
  'id': 10,  
  'nama': 'Botol Plastik',  
  'harga': 5000,  
  'tersedia': true,  
};
```

String adalah tipe key, sedangkan **dynamic** berarti value dapat memiliki tipe berbeda.

Dalam aplikasi Flutter, **Map** sering digunakan untuk:

- Membaca JSON dari backend.
- Mengirim request ke API.
- Menyimpan konfigurasi sederhana.

Contoh request body:

```
Map<String, dynamic> requestBody = {  
  'email': 'budi@example.com',  
  'password': 'rahasia',  
};
```

Untuk aplikasi yang lebih besar, data **Map** sebaiknya diubah menjadi class/model agar lebih aman.

9. Object — induk dari hampir semua objek Dart

Object dapat menyimpan objek dengan tipe berbeda:

```
Object data = 'Budi';  
data = 20;  
data = true;
```

Namun, sebelum menggunakan operasi khusus, tipenya perlu diperiksa:

```
if (data is String) {  
  print(data.toUpperCase());  
}
```

Gunakan **Object** ketika sebuah nilai dapat memiliki beberapa tipe, tetapi Anda masih ingin melakukan pemeriksaan tipe secara aman.

10. *dynamic* — tipe dapat berubah-ubah

dynamic membuat Dart tidak memeriksa operasinya secara ketat saat proses kompilasi.

```
dynamic data = 'Budi';  
data = 20;  
data = true;
```

Contoh:

```
dynamic nama = 'Budi';  
print(nama.toUpperCase());
```

Masalahnya, kode berikut dapat lolos dari pemeriksaan awal tetapi gagal saat dijalankan:

```
dynamic nama = 100;  
print(nama.toUpperCase());
```

Angka tidak memiliki fungsi *toUpperCase()*, sehingga terjadi runtime error.

Gunakan *dynamic* secara terbatas, biasanya saat:

- Membaca JSON.
- Tipe data dari sumber luar belum diketahui.
- Berinteraksi dengan library tertentu.

Jika tipenya sudah diketahui, pilih tipe yang spesifik.

```
String nama = 'Budi'; // Lebih aman
```

3

Immutability & Lifecycle Variabel

Perbedaan **final** vs **const**:

- **final**: Nilai dievaluasi saat *runtime* (misal: `DateTime.now()`), ID transaksi dinamis) dan dikunci satu kali.
- **const**: Nilai tetap saat *compile-time* (misal: tarif pajak tetap, kode mata uang).

Penggunaan **late**: Inisialisasi variabel non-nullable yang nilainya baru tersedia sebelum dibaca.

Konsep **Immutability** (data tidak dapat diubah setelah dibuat) adalah fondasi performa Flutter, terutama saat mengelola state dan widget tree.

Immutability adalah fondasi performa antarmuka Flutter.

Kata Kunci	Waktu Evaluasi (Evaluation Time)	Sifat di Memori	Contoh Penggunaan
final	Runtime (saat program berjalan)	Diinisialisasi sekali, setelah itu nilainya terkunci mutlak.	ID unik transaksi, jam sistem (<code>DateTime.now()</code>).
const	Compile-Time (sebelum program berjalan)	Alokasi memori kanonikal (hemat memori). Nilai harus konstan mutlak.	PPN tetap (0.11), kode mata uang (IDR).
late	Inisialisasi Tertunda [cite: 1]	Tipe non-nullable yang nilainya baru disuplai sebelum pemanggilan pertama[cite: 1].	Pengisian data nomor nota transaksi setelah kalkulasi selesai[cite: 1].

Contoh kasus :

```
void main() {  
    // Runtime: jam saat fungsi dipanggil belum diketahui saat compile-time  
    final DateTime createdAt = DateTime.now();  
    // createdAt = DateTime.now(); // Error! final tidak bisa di-reassign.  
  
    // Compile-time: nilai sudah pasti mutlak  
    const double taxRate = 0.11; // PPN 11%  
    // taxRate = 0.12; // Error!  
}
```

Penerapan Immutable Class: Untuk membuat class yang aman dari perubahan data liar, semua *fields* ditandai sebagai **final**. Jika ingin membuat konstruktor **const**, seluruh properti wajib bertipe **final**.

```
class CurrencyFormatter {  
    final String symbol;  
  
    // const constructor memungkinkan objek dibuat saat compile-time  
    const CurrencyFormatter({required this.symbol});  
  
    String format(double value) => '$symbol ${value.toStringAsFixed(0)}';  
}
```


5

Assignment

Installation (Flutter SDK , IDE , Android Tooling , Git)

Install Flutter SDK

- ☐ Flutter SDK sudah terpasang.
- ☐ Flutter sudah ditambahkan ke PATH.
- ☐ flutter --version berhasil.
- ☐ dart --version berhasil.
- ☐ flutter doctor berhasil dijalankan.

Install IDE

- ☐ Visual Studio Code atau Android Studio terpasang.
- ☐ Extension Flutter terpasang.
- ☐ Extension Dart terpasang.
- ☐ Menu Flutter: New Project tersedia.

Install Android Tooling

- ☐ Android Studio terpasang.
- ☐ Android SDK terpasang.
- ☐ Android SDK Command-line Tools terpasang.
- ☐ Android SDK Platform-Tools terpasang.
- ☐ Lisensi Android sudah diterima.
- ☐ Emulator atau Android fisik terdeteksi.
- ☐ flutter devices menampilkan minimal satu perangkat.

Install Git & Register Repository (Github)

- ☐ Git command
- ☐ Registrasi Git
- ☐ Commit dan push code pada latihan pada dartPad

Validasi akhir

Jalankan seluruh perintah berikut:

```
git --version  
flutter --version  
dart --version  
flutter doctor  
flutter emulators  
flutter devices
```

Target akhir Assignment:

Tidak ada error Flutter.

Android toolchain tersedia.

Visual Code mendeteksi Flutter.

Minimal satu emulator atau perangkat Android tersedia.

Mahasiswa siap membuat project pada Pertemuan 2.